

httpx Tool - Tester Workflow Report

Step-by-Step Execution Process for Reconnaissance Testing

Purpose

This report documents the standard step-by-step process a security tester follows when using **httpx** for reconnaissance (live host/service probing), from initial setup through to final reporting. Each step includes the situation in which it applies and the corresponding command, followed by a decision guide for choosing between options mid-engagement.

What Is httpx?

httpx is a fast, open-source HTTP toolkit developed by ProjectDiscovery. Where Subfinder and Amass answer the question “*what subdomains/assets exist?*”, httpx answers the next question: “*which of these are actually live web services, and what do they look like?*” It takes a list of hosts/subdomains and probes them over HTTP/HTTPS, reporting status codes, titles, technologies, response headers, and more. It’s almost always run **after** subdomain enumeration, as the bridge between “asset discovery” and “deeper testing.”

Step 1: Scoping and Authorization

Situation: Before probing any hosts, the tester confirms the domains/IPs being fed into httpx are covered by the same authorization as the earlier enumeration phase. Since httpx sends live requests to targets, this step matters even more here than in passive-only tools.

Action: No command yet — confirm the input list only contains in-scope hosts, and confirm whether any hosts should be excluded (e.g., a client’s production payment system).

Step 2: Basic Live-Host Probing

Situation: This is the **first httpx run**, typically right after subdomain enumeration (Subfinder/Amass) has produced a list of candidate hosts. The tester wants to quickly see which ones actually respond.

Command:

```
httpx -l subdomains.txt
```

What it does: Reads a list of hosts, sends HTTP/HTTPS requests to each, and prints only the ones that return a valid response — filtering out dead or non-existent hosts.

Step 3: Capturing Status Codes and Titles

Situation: A plain list of live URLs isn’t very informative on its own. The tester wants basic context on each one — is it a login page? A default server page? An error?

Command:

```
httpx -l subdomains.txt -status-code -title -o results.txt
```

When to use it: Almost always — this is the standard “readable” output level for a client-facing report or a quick manual review.

Step 4: Technology Fingerprinting

Situation: The tester wants to know what’s running behind each live host (web server, CMS, framework) to help prioritize which targets are worth deeper testing.

Command:

```
httpx -l subdomains.txt -tech-detect -o results.txt
```

Situation-specific note: Useful for quickly spotting outdated or known-vulnerable software versions across a large list of hosts.

Step 5: Structured Output for Automation

Situation: Results need to be passed into another tool (a vulnerability scanner like Nuclei, a screenshot tool, or a custom script) rather than read by a person.

Command:

```
httpx -l subdomains.txt -json -o results.json
```

When to use it: Any time httpx sits in the middle of an automated pipeline, e.g. `subfinder -> httpx -> nuclei`.

Step 6: Filtering by Response Code

Situation: A large host list often includes many uninteresting responses (redirects, 404s, default pages). The tester wants to focus only on hosts that returned a meaningful response.

Command:

```
httpx -l subdomains.txt -mc 200,301,302 -o results.txt
```

When to use it: When the list is large and the tester needs to cut noise before manual review — `-mc` (match code) keeps only the specified status codes.

Step 7: Screenshotting Live Hosts

Situation: The tester needs to visually review a large number of hosts quickly — for example, to spot login panels, admin dashboards, or obviously misconfigured pages — without opening each one manually.

Command:

```
httpx -l subdomains.txt -screenshot -o screenshots/
```

When to use it: Larger engagements with hundreds of hosts, where visual triage is faster than reading text output line by line.

Step 8: Controlling Request Rate

Situation: The client's infrastructure is sensitive to request volume, or the scope of work specifies a maximum request rate to avoid any risk of disruption.

Command:

```
httpx -l subdomains.txt -rate-limit 50 -o results.txt
```

Situation-specific note: `-rate-limit` caps requests per second; always checked against the client's agreed-upon limits before running at scale.

Step 9: Documenting and Reporting Results

Situation: The final step in every engagement. The tester records:

- Exact command(s) and flags used
 - Number of hosts probed vs. number that responded live
 - Status codes, titles, and technologies observed
 - Any hosts excluded from scope
 - Output file(s) ready to hand off to the next phase (vulnerability scanning, manual review, screenshots for the client report)
-

Decision Guide — Which Command Does the Tester Choose, and Why?

At several points in an engagement, the tester has to make a judgment call about which httpx command or option to run next. This section covers those decision points.

Situation the Tester Encounters	Decision & Reasoning	Command Used
Just received a subdomain list and wants to quickly see what's actually alive	Run the simplest possible probe with no extra flags	<code>httpx -l subdomains.txt</code>
Live host list is ready, but the tester needs basic context on each one	Add status code and title flags for a human-readable overview	<code>httpx -l subdomains.txt -status-code -title -o results.txt</code>
Tester needs to prioritize which hosts are worth manual testing first	Add technology detection to spot outdated or high-value software	<code>httpx -l subdomains.txt -tech-detect -o results.txt</code>
Output will be consumed by another automated tool, not read by a person	Switch to structured JSON output instead of plain text	<code>httpx -l subdomains.txt -json -o results.json</code>
Host list is very large and full of uninteresting redirects/404s	Filter to only the status codes that matter before manual review	<code>httpx -l subdomains.txt -mc 200,301,302 -o results.txt</code>
Engagement has hundreds of live hosts and manual review would take too long	Generate screenshots for fast visual triage instead of reading text line by line	<code>httpx -l subdomains.txt -screenshot -o screenshots/</code>
Client's scope of work specifies a maximum request rate	Cap requests per second to stay within the agreed-upon limit	<code>httpx -l subdomains.txt -rate-limit 50 -o results.txt</code>
Tester wants to feed only currently-live hosts into a vulnerability scanner next	Use httpx's clean/silent output directly as input to the next tool in the chain	<code>httpx -l subdomains.txt -silent -o live.txt</code>
Some hosts in the list turn out to be out of scope (discovered mid-scan)	Stop, remove those hosts from the input file, and re-run rather than probing them	<i>(no command — list must be edited before re-running)</i>
Tester wants to confirm results are consistent before finalizing the report	Re-run the same probe a second time and compare, since live-web state can change between scans	<code>httpx -l subdomains.txt -status-code -title -o results_v2.txt</code>

General decision principle the tester follows: Because httpx sends real, live traffic to every host it probes, the main judgment calls are about *scope discipline* (never probing excluded hosts) and *request rate* (staying within what the client agreed to), in addition to choosing the right output detail level for whoever consumes the results next — a person reviewing manually, or another tool further down the pipeline.

Summary Table — Full Tester Journey

#	Step	Situation	Example Command
1	Scoping & authorization	Before any probing	<i>(no command — approval step)</i>
2	Basic live-host probing	First run, right after enumeration	<code>httpx -l subdomains.txt</code>
3	Status codes & titles	Need human-readable context	<code>httpx -l subdomains.txt -status-code -title -o results.txt</code>
4	Technology fingerprinting	Prioritizing targets	<code>httpx -l subdomains.txt -tech-detect -o results.txt</code>
5	Structured JSON output	Feeding into another tool	<code>httpx -l subdomains.txt -json -o results.json</code>
6	Filter by response code	Cutting noise from a large list	<code>httpx -l subdomains.txt -mc 200,301,302 -o results.txt</code>
7	Screenshotting	Visual triage at scale	<code>httpx -l subdomains.txt -screenshot -o screenshots/</code>
8	Controlling request rate	Client rate-limit requirements	<code>httpx -l subdomains.txt -rate-limit 50 -o results.txt</code>
9	Reporting	Always, at the end	This report

This report describes standard httpx usage for the live-host probing phase, typically run right after subdomain enumeration (Subfinder/Amass) and before deeper testing, and only ever against hosts that are explicitly in scope and authorized for testing.